Understanding Codebase

① Diffusion Models

1. The Generative Modeling Problem
suppose we have the dataset of real samples: $x_0 \sim P_{data}(x)$ where x_0 = data sample & $P_{data}(x)$ = the true data distribution
our goal is to learn a model that generates samples from the same distribution. $P_\theta(x)$ where P_θ = model distribution and θ = neural network parameters

2. Core Idea of Diffusion Models
Instead of generating data directly, diffusion models do two things: 1) Define a process that gradually destroys data with noise 2) Train a neural network to reverse that process
Forward Process: $x_0 \to x_1 \to x_2 \dots \to x_T$
Eventually $x_T \sim N(0, I)$ pure gaussian noise
Generation works by reversing this: $x_T \to x_{T-1} \to \dots \to x_0$ so we learn how to progressively remove noise

3. Forward Diffusion Process
we define a Markov chain that gradually adds noise. A Markov chain means $q(x_t | x_{t-1})$ the next state depends only on the previous one.
we define the transition: $x_t = \sqrt{1-\beta_t}\, x_{t-1} + \sqrt{\beta_t}\, \varepsilon$ where $\varepsilon \sim N(0,1)$ Distribution Form: $q(x_t | x_{t-1}) = \mathcal{N}(\underbrace{\sqrt{1-\beta_t}\, x_{t-1}}_{\text{mean}}, \underbrace{\beta_t I}_{\text{variance}})$

noisier sample → x_t; β_t = noise schedule; x_{t-1} = previous sample; $\sqrt{\beta_t}$ = noise schedule; ε = Gaussian noise

4. Noise Schedule
Parameters β_t control how quickly noise increases
Typical values:
$\beta_1 \approx 10^{-4}$
$\beta_T \approx 0.02$
Early steps add very little noise.

5. Skip-Step Formula
Instead of applying noise step by step, we derive a formula for sampling x_t directly from x_0.
First define: $\alpha_t = 1 - \beta_t$ where α_t is how much signal remains after one step
Define cumulative product: $\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s$. This represents how much original signal remains after t steps.
Then, we can show $x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\, \varepsilon$ where $\varepsilon \sim N(0,1)$
A noisy sample consists of a signal component and noise component

6. The Reverse Process
we want to learn the reverse transition: $P_\theta(x_{t-1} | x_t)$ means given a noisy sample predict a slightly cleaner example.
The reverse distribution can be modeled as Gaussian: $P_\theta(x_{t-1} | x_t) = \mathcal{N}(\underbrace{\mu_\theta(x_t, t)}_{\text{mean}}, \underbrace{\Sigma_\theta(x_t, t)}_{\text{variance}})$

neural network predicts the most likely x_{t-1}
most implementations only predict the noise: $\varepsilon_\theta(x_t, t)$
this is fixed per timestamp

7. True Reverse Distribution
$q(x_{t-1} | x_t, x_0)$ It depends on the original clean sample x_0. But during generation, we do not know x_0 - the model must estimate it.

8. Recovering Clean Data from Noise
Recall the forward equation: $x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\, \varepsilon$ If we know the noise inside a sample, we can reconstruct the clean data.
solve for x_0, gives $x_0 = \frac{1}{\sqrt{\bar{\alpha}_t}} (x_t - \sqrt{1-\bar{\alpha}_t}\, \varepsilon)$

9. Neural Network Prediction
The model predicts the noise: $\varepsilon_\theta(x_t, t)$

x_t = noisy sample; ε_θ = predicted noise

10. Training Procedure
Algorithm
1) sample real data $x_0 \sim P_{data}$
2) sample timestep $t \sim \text{uniform}(1, T)$
3) sample noise $\varepsilon \sim N(0, I)$
4) generate noisy sample $x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\, \varepsilon$
5) train model to predict noise $L = E[\|\varepsilon - \varepsilon_\theta(x_t, t)\|^2]$

11. Inference
Generation starts with random noise: $x_T \sim N(0, I)$
Then, we repeatedly apply the reverse transition
The update step:

$$x_{t-1} = \underbrace{\frac{1}{\sqrt{\alpha_t}}}_{\text{undo forward scaling}} \underbrace{\left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\, \varepsilon_\theta(x_t, t)\right)}_{\text{remove the noise from } x_t} + \underbrace{\sigma_t z}_{\text{adds fresh random noise}} \quad \text{where } z \sim N(0,1)$$

② Flow Matching Notes

1. The Generative Modeling Problem
   Suppose we have real samples $x_1 \sim P_{data}(x)$
   Use x_1 for the data endpoint here because in flow matching it is often convenient to let: $t=0$ correspond to noise & $t=1$ correspond to data
   x_0 will often mean a sample from the source distribution (gaussian noise)
   x_1 will mean a sample from the target distribution (real data)
   Our goal is to learn a model that can transform samples from a simple distribution, like Gaussian noise $x_0 \sim P_0(x) = N(0, I)$ into sample from the data distribution $x_1 \sim P_1(x) = P_{data}(x)$

2. Core Idea of Flow Matching
   Let's define a continuous path that moves samples from the source distribution P_0 to the target distribution P_1, and learn the velocity field of that motion.
   noise $\longrightarrow$ move continuously thru space $\rightarrow$ data
   model learns $v_\theta(x,t)$ where x = current point, $t \in [0,1]$ is time along path, $v_\theta(x,t)$ is velocity vector telling us how the point should be.

3. Defining a Probability Path
   We want a whole family of intermediate distributions $p_t(x), t \in [0,1]$ s.t. at $t=0, P_t = P_0$ = noise distribution & at $t=1, P_t = P_1$ = data distribution
   The family P_t is called a probability path
   At $t=0$, cloud looks Gaussian
   At $t=1$, cloud looks like the data distribution
   Flow matching learns the vector field that moves this cloud correctly.

4. Interpulating Between Noise & Data
   We need examples of points x_t that lie somewhere along the path between noise & data. Simple & most common choice is linear interpolation.
   $x_0 \sim P_0$, usually Gaussian noise
   $x_1 \sim P_1$, real data sample
   $x_t = (1-t)x_0 + tx_1$ At $t=0: x_t = x_0$ & At $t=1: x_t = x_1$ small t = mostly noise & large t = mostly data

5. The Velocity Field
   If a point is moving along the path x_t, what is its velocity?
   Take the derivative of the interpolation equation with respect to time: $x_t = (1-t)x_0 + tx_1 \rightarrow \frac{dx_t}{dt} = -x_0 + x_1 = x_1 - x_0$ $u_t = x_1 - x_0$

6. Conditional Velocity Field
   More generally, if we condition on a pair (x_0, x_1), we can define a path $x_t = \phi_t(x_0, x_t)$ & a corresponding conditional velocity field: $u_t(x_t | x_0, x_1)$
   For linear interpolation: $\phi(x_0, x_1) = (1-t)x_0 + tx_1$ & $u_t(x_t | x_0, x_1) = x_1 - x_0$
   During training, we know both x_0 and x_1 so we can compute this target velocity exactly
   During generation, we only have the current point x_t, so the model must infer the right velocity from x_t & t

7. The Model
   We train a neural network $v_\theta(x,t)$ to approximate the correct velocity field.

8. The Training Target
   For linear interpolation, the true velocity is $u_t = x_1 - x_0$. So given a training pair (x_0, x_1), we can sample a time t, compute the point $x_t = (1-t)x_0 + tx_1$ and then supervise the neural network with the target $x_1 - x_0$.

9. The Learning Objective
   $L_{FM} = E_{x_0, x_1, t}\left[\|v_\theta(x_t, t) - (x_1 - x_0)\|^2\right]$ ← sample many source points x_0, many data points x_1, many times t & average the squared error
   Model is rewarded when its predicted velocity matches the true path velocity

10. Why this works?
    Once the model has learned the velocity field, we can generate a sample by solving the ODE
    $$\frac{dx}{dt} = v_\theta(x,t)$$
    Starting from noise: $x(0) \sim P_0 = N(0, I)$ then we integrate forward from $t=0$ to $t=1$
    If the learned vector field is correct, the final point $x(1)$ will be distributed like the data.

11. The Generative Process
    Start with a random point from the source distribution: $x(0) \sim N(0,1)$. Then evolve it using the learned ODE: $\frac{dx}{dt} = v_\theta(x,t)$ until $t=1$

12. Full Training Procedure
    <u>Algorithm</u>
    1) sample source noise $x_0 \sim N(0, I)$
    2) sample real data $x_1 \sim P_{data}$
    3) sample time $t \sim \text{uniform}(0,1)$
    4) construct an intermediate point $x_t = (1-t)x_0 + tx_1$
    5) compute the target velocity $u_t = x_1 - x_0$
    6) Train the model to predict that velocity $L = \|v_\theta(x_t, t) - u_t\|^2$

13. Inference
    We solve the ODE numerically
    $x_{t+\Delta t} = x_t + \Delta t\, v_\theta(x_t, t)$ where Δt is a small step size
    1) look at current point x_t
    2) evaluate the velocity field there
    3) move a little in that direction
    4) repeat